

AI AGENT FOR PLASTIC WASTE SEGREGATION

USING COMPUTER VISION

1. Introduction

Waste segregation is an important step in improving recycling and reducing environmental pollution. This project develops a computer vision-based AI agent that automatically identifies plastic, wet, and dry waste and directs each item to the correct collection bin.

2. PEAS Table

Component	Details
Performance Measure	The agent should accurately classify waste and minimize incorrect segregation while maintaining fast processing speed.
Environment	The system operates in waste collection centers where different types of waste move on a conveyor belt.
Actuators	Robotic arms, conveyor belt diverters, and pneumatic air jets are used to place waste into the appropriate bins.
Sensors	RGB cameras, proximity sensors, and weight sensors capture images and detect the presence of waste.

3. Environment Properties

Property	Explanation
Observability	Partially Observable: The camera may not always capture a complete view of every waste item.
Determinism	Stochastic: Lighting conditions and overlapping waste can affect classification results.

Dynamics	Dynamic: The environment changes continuously as new waste items arrive.
Continuity	Continuous: Waste moves continuously, and the robotic arm operates with continuous movements.
Episodic vs Sequential	Sequential: Each sorting decision influences the next item moving on the conveyor belt.
Number of Agents	Single Agent: Only one AI agent performs the waste classification and sorting.
Known vs Unknown	Known: The agent is trained to recognize predefined waste categories and system behavior.

4. Rationality Measure

The AI agent acts rationally by analyzing camera images, classifying the waste using a trained computer vision model, and selecting the correct sorting action. It maximizes its performance by reducing classification errors and ensuring each waste item is placed into the correct bin.

5. Innovation Point

This system uses edge AI to perform real-time waste classification without relying on cloud connectivity. It also supports dynamic bin load balancing to distribute waste efficiently and reduce overflow.

6. Diagram Agent Architecture

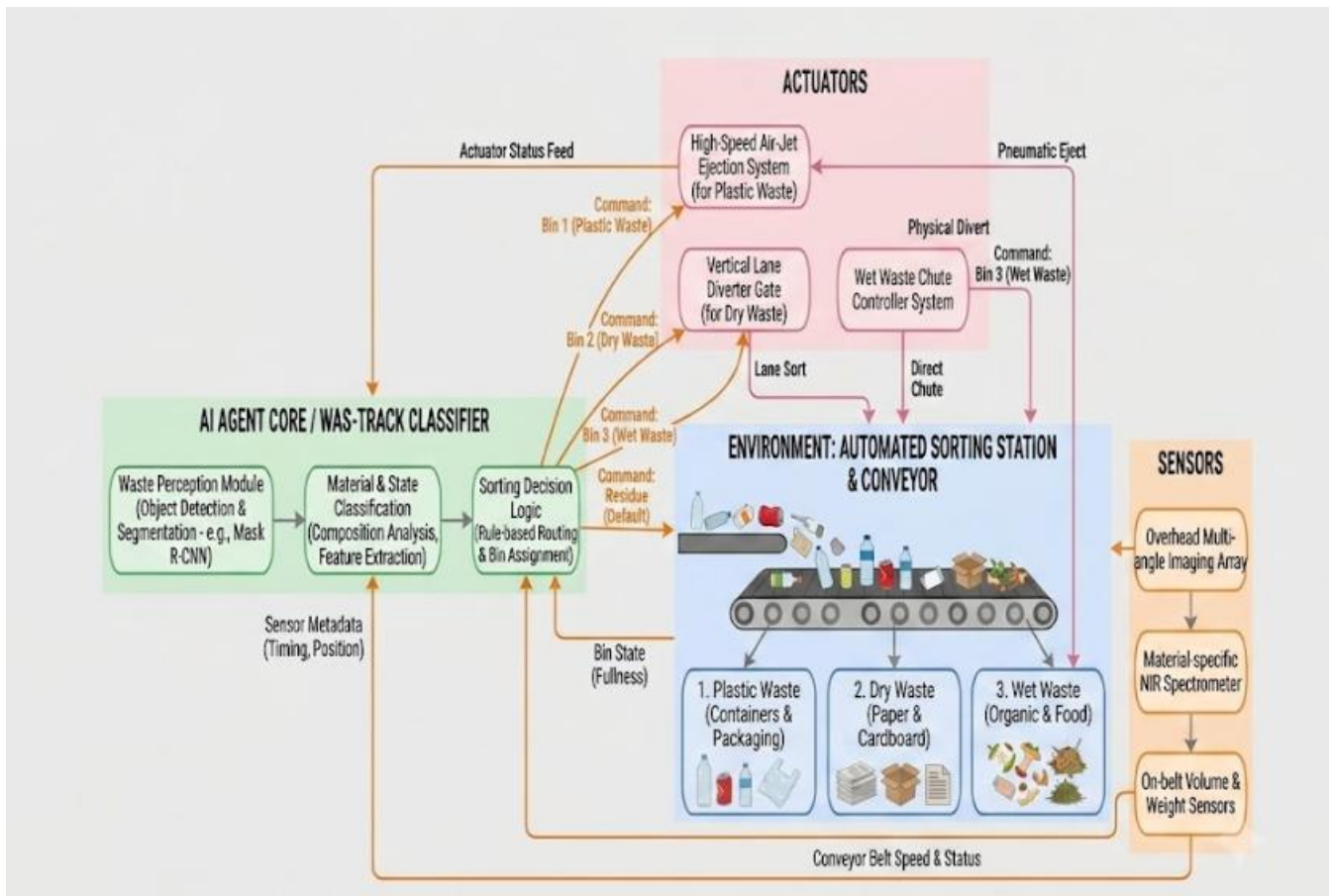


Fig 1. Waste Detection → Material Classification → Sorting Logic → Action pipeline linking cameras, spectral sensors, and actuators to sorted collection bins (Wet, Dry, Plastic). Includes multiple feedback loops for process control and status monitoring.

7. Conclusion

The proposed AI agent improves waste segregation accuracy, reduces manual effort, and supports efficient recycling processes. In the future, advanced sensors and deep learning models can further enhance classification accuracy and enable sorting of additional waste categories.

8. Simulation

The simulation demonstrates the working of the AI-based waste segregation agent in a virtual environment. It begins by simulating the capture of waste images using a camera. Instead of using a real camera feed, the program generates or loads sample frames representing different waste items.

Each frame is then processed by the computer vision module, which analyzes the image and classifies the waste into one of three categories: plastic, wet, or dry. The classification can be performed using a trained deep learning model (such as a CNN) or by using mock computer vision logic for simulation purposes.

After the waste type is identified, the decision-making module selects the appropriate sorting action. Based on the classification result, the program generates actuation commands such as moving the robotic arm to a specific angle, activating a conveyor belt diverter, or triggering a pneumatic air jet to direct the waste into the correct collection bin.

The simulation continuously repeats this process for multiple waste items, displaying the detected waste type, confidence score (if applicable), selected bin, and corresponding actuator command. This demonstrates the complete workflow of the intelligent waste segregation agent, from image acquisition to automated waste sorting, without requiring physical hardware.

Flow of the Simulation:

Capture Waste Image → Process Image → Classify (Plastic/Wet/Dry) → Generate Actuation Command → Move Waste to Correct Bin → Display Output → Repeat for Next Waste Item

9. Full Code and Output

9.1 Full Code

```
import cv2
import numpy as np
import tensorflow as tf

# Load the TFLite model from your inner folder path
interpreter = tf.lite.Interpreter(model_path="converted_tflite/model_unquant.tflite")
interpreter.allocate_tensors()

# FIX: Get the first item [0] from the details list
input_details = interpreter.get_input_details()[0]
output_details = interpreter.get_output_details()[0]

# Load the labels file from your inner folder path
class_names = open("converted_tflite/labels.txt", "r").readlines()

# Open your PC's default webcam (0 is usually the built-in webcam)
camera = cv2.VideoCapture(0)
```

```
print("Starting local TFLite Waste Sorting... Press 'ESC' to exit.")
```

```
while True:
```

```
    # Grab a frame from the webcam
```

```
    ret, image = camera.read()
```

```
    if not ret:
```

```
        print("Failed to grab frame from webcam. Exiting...")
```

```
        break
```

```
    # Resize image frame to match model's expected shape input (224x224)
```

```
    frame = cv2.resize(image, (224, 224), interpolation=cv2.INTER_AREA)
```

```
    # Convert image matrix to numpy array and shape format
```

```
    frame = np.asarray(frame, dtype=np.float32).reshape(1, 224, 224, 3)
```

```
    # Normalize image array to match Teachable Machine's training preprocessing
```

```
    frame = (frame / 127.5) - 1
```

```
    # Inject image frames into the model and process using the correct list index mapping
```

```
    interpreter.set_tensor(input_details['index'], frame)
```

```
    interpreter.invoke()
```

```
    # Retrieve output predictions using the correct list index mapping
```

```
    prediction = interpreter.get_tensor(output_details['index'])[0]
```

```
    index = np.argmax(prediction)
```

```
    class_name = class_names[index].strip()
```

```
    confidence_score = prediction[index]
```

```
    # Draw the detected waste classification label directly onto live video feed
```

```
    display_text = f'Class: {class_name} ({np.round(confidence_score * 100, 2)}%)'
```

```
    cv2.putText(image, display_text, (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2,  
cv2.LINE_AA)
```

Display the live window

```
cv2.imshow("Local Waste Sorting AI", image)
```

Listen for the Escape key (27) to cleanly close down the script

```
keyboard_input = cv2.waitKey(1)
```

```
if keyboard_input == 27:
```

```
    break
```

Clean up memory and release camera stream

```
camera.release()
```

```
cv2.destroyAllWindows()
```

```
print("Webcam closed down successfully.")
```

9.2 Output-Screenshots

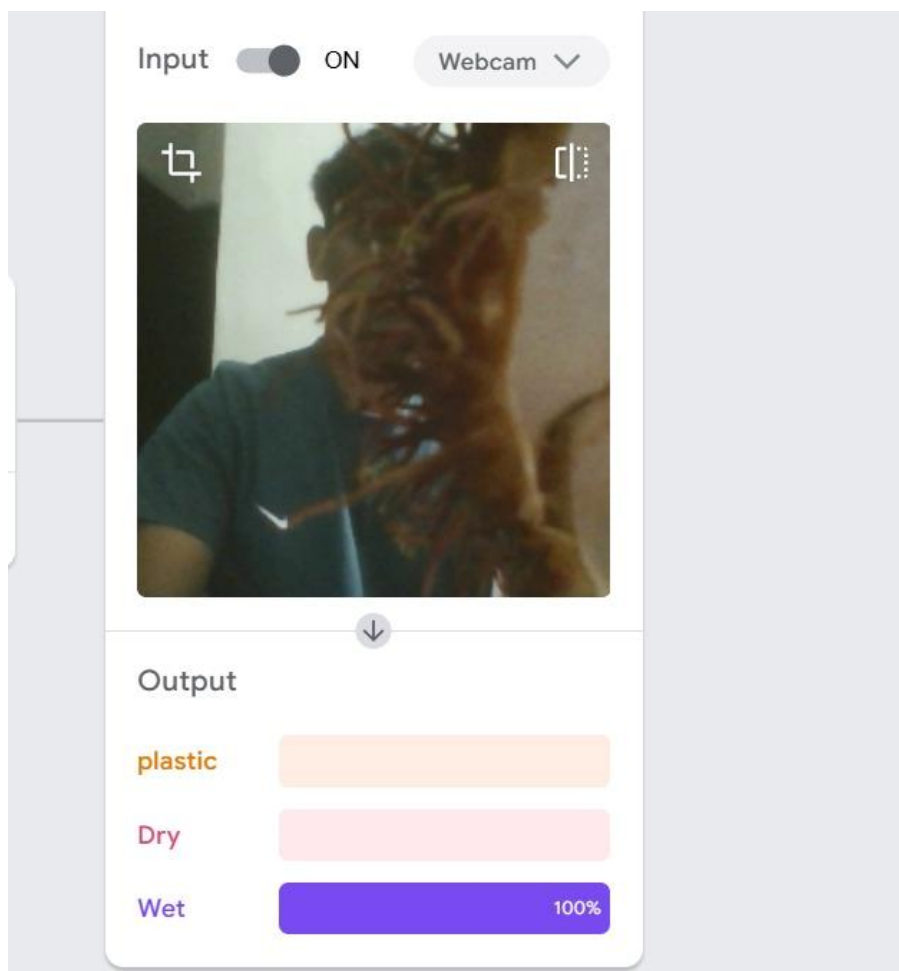


Fig 2. Waste is wet

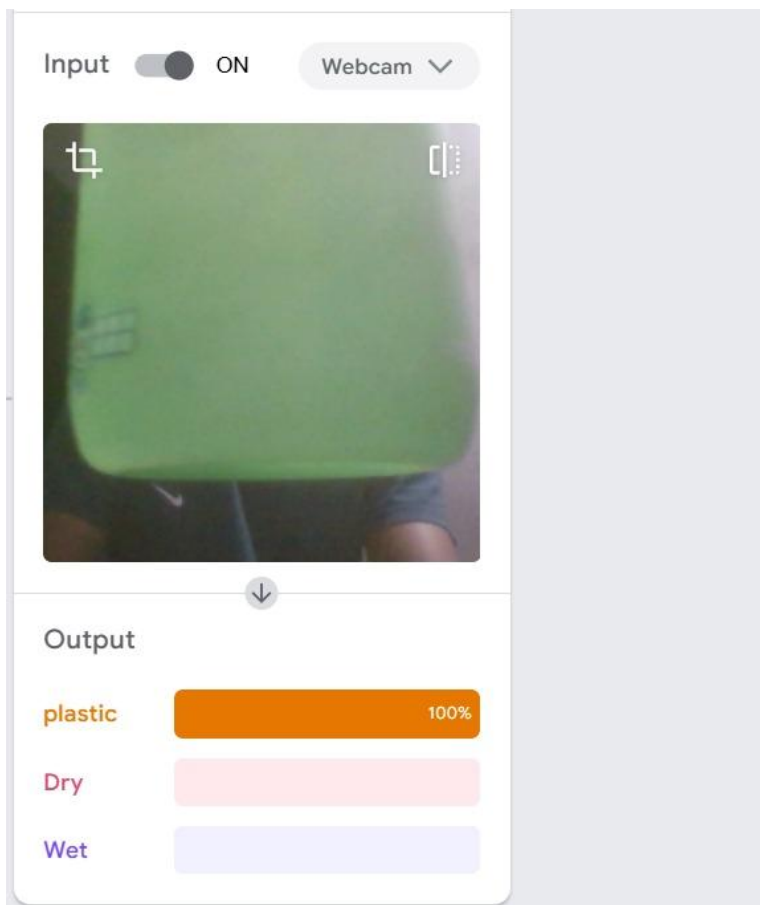


Fig 3. Waste is plastic

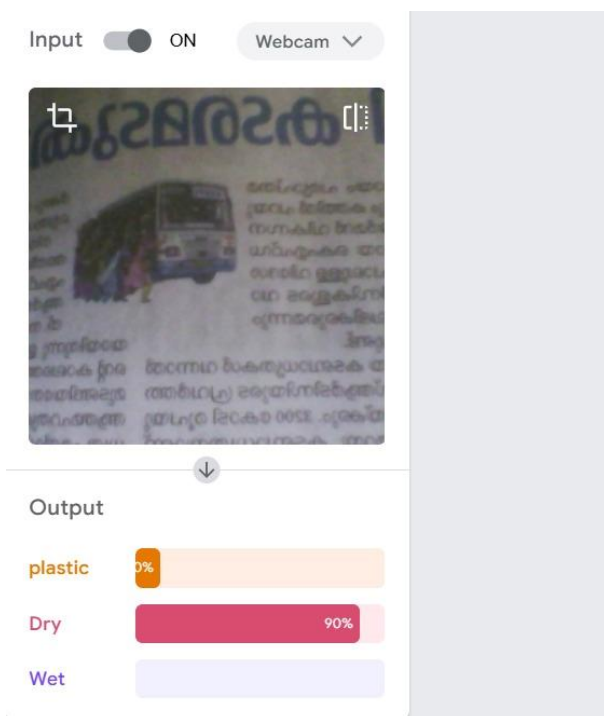


Fig 4. Waste is dry